AR-DSA Tutor: A Web-Based Spatial Algorithm Visualization System with Execution-Grounded Explainable Adaptation

Ajay A
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology (VIT), Chennai, Tamil Nadu, India
contact: ajay.a2023@vitstudent.ac.in

Abstract*—Modern computer science education continues to struggle with the representational disconnect and cognitive split-attention induced when learners attempt to map dynamic, multi-pointer data structure invariants (such as binary search tree rebalancing, logarithmic search boundary halving, and breadth-first search queue frontiers) onto passive, two-dimensional textbook diagrams. Meanwhile, recent efforts to integrate conversational artificial intelligence (AI) tutors introduce unpredictable network latency (1.5s–3.5s), spatial hallucination, and mathematically invalid advice that violates formal algorithm invariants. In this paper, we develop and verify AR-DSA Tutor, an interactive, zero-install mobile WebXR system that renders five distinct algorithm paradigms—Binary Search Tree, Bubble Sort, Binary Search, Breadth-First Search, and Stack—as interactive, spatially anchored 3D models running directly within commodity mobile web browsers. To scaffold learning without violating the 16.6ms frame budget required for 60 FPS mobile WebXR, we engineer Execution-Grounded Explainable Adaptation (EGEA), a client-side deterministic policy engine that computes pedagogical interventions in steady-state median time of 7 µs (mean 10 µs, P95 20 µs), with complete ingestion and adaptation pipeline latency of 14 µs (mean 17 µs, P95 24 µs). EGEA couples an exponentially weighted moving average (EWMA) temporal pacing model with a six-tier scaffolding taxonomy, successfully disentangling impulsive trial-and-error guessing (action latency < 1500ms) from deliberate conceptual reflection and autonomous mastery streaks. Evaluating across an authored 18-case engineering challenge suite and a 12-case authored boundary stress suite, EGEA achieves 100.0% decision consistency, outperforming stateless fixed-hint (33.3%) and error-rule (50.0%) baselines while resolving four documented boundary failure modes. The complete system compiles into a 319 kB gzipped client bundle, supports seamless algorithm hot-swapping within a single continuous WebXR session, and passes all 18 automated verification test suites across 1,437 assertions.

Index Terms*—Augmented reality in education, WebXR, spatial algorithm visualization, intelligent tutoring systems, explainable adaptation, mobile spatial computing, deterministic state machines, data structures and algorithms.

I. INTRODUCTION AND PROBLEM STATEMENT

Instruction in abstract Data Structures and Algorithms (DSA) represents a cornerstone of undergraduate computer science curricula. Mastery of algorithmic invariants—such as logarithmic interval contraction, tree ordering constraints, and FIFO queue state transitions—requires learners to construct robust mental models of dynamic memory relationships. However, decades of educational research establish that students experience severe cognitive hurdles when learning DSA through conventional instructional media [1], [2].

This pedagogical challenge is driven by two acute structural bottlenecks in existing educational software:

1) The Representational Disconnect in 2D Viewports: Classical algorithm animation platforms, such as VisuAlgo [3] and OpenDSA [4], project multi-pointer data structures onto flat 2D computer screens. This forces learners into a high-overhead cognitive mapping task known as the split-attention effect [2]. Learners must mentally reconstruct three-dimensional depth, parent-child hierarchical relations, and spatial traversal frontiers while simultaneously parsing textual code listings. Furthermore, passive animation viewing induces the illusion of explanatory depth, where students overestimate their understanding without acquiring operational problem-solving competence [1].

2) The Scaffolding Dilemma: Brittle Static Rules vs. Hallucinatory LLMs: Current automated tutoring systems generally vacillate between two undesirable extremes. On one hand, static rule-based checkers offer unadaptive, hard-coded hints that ignore learner pacing, impulsive trial-and-error guessing, and demonstrated mastery. On the other hand, recent deployments of conversational Large Language Models (LLMs) suffer from prohibitive network round-trip latencies (1.5s–3.5s) that disrupt mobile rendering loops, alongside well-documented tendencies to hallucinate non-existent algorithmic properties or provide mathematically incorrect advice [5].

To resolve these fundamental limitations, we present AR-DSA Tutor, an interactive, zero-install mobile WebXR educational system. Rather than requiring proprietary hardware or multi-hundred-megabyte native application downloads, AR-DSA Tutor runs directly in standard mobile web browsers via the W3C WebXR Device API [6]. The platform anchors abstract algorithmic models onto physical tabletops, enabling learners to physically walk around, inspect, and manipulate discrete data structures [7] without exceeding the 16.6ms render budget of commodity mobile devices [8].

The primary scientific and systems contributions of this paper are organized as follows:
• Unified Multi-Algorithm Spatial WebXR Architecture: We engineer a persistent session architecture that hosts five foundational algorithms (Binary Search Tree, Bubble Sort, Binary Search, Breadth-First Search, and Stack) within a single WebXR session, permitting instant topic switching without camera feed interruption or GPU context teardown.
• Execution-Grounded Explainable Adaptation (EGEA): We introduce a sub-millisecond deterministic policy engine that executes pure intervention logic in steady-state median latency of 7 µs (mean 10 µs, P95 20 µs) and complete end-to-end telemetry ingestion in 14 µs (mean 17 µs, P95 24 µs), consuming < 0.15% of a 60 FPS mobile frame budget while producing 100% auditable pedagogical rationales.
• Pacing-Aware Guessing Throttling: Coupling an Exponentially Weighted Moving Average (EWMA) temporal model with multi-tier scaffolding, our system differentiates rapid trial-and-error clicking (latency < 1500ms) from deliberate reflection, dynamically enforcing deliberate pauses and fading scaffolding upon verified independent mastery.

• Rigorous Verification and Open Release: The application compiles into a lightweight 319 kB gzipped bundle, satisfies 1,437 automated test assertions across 18 verification suites, and is evaluated across an authored 18-case engineering challenge suite and a separate 12-case boundary stress suite.

II. RELATED WORK AND RESEARCH GAP

A. *Spatial and 3D Algorithm Visualization*

Algorithm visualization has a rich history dating to early graphical environments like Balsa-II [9] and Tango [10]. Modern web-based systems, notably VisuAlgo [3] and OpenDSA [4], demonstrated that interactive self-paced animations improve student engagement, as synthesized in comprehensive reviews by Sorva et al. [11]. However, these systems remain confined to flat 2D viewports. In contrast, early 3D visualization experiments frequently induced visual clutter and disorientation due to arbitrary 3D camera navigation. AR-DSA Tutor overcomes this by physically anchoring structures to real-world surfaces with fixed upright (+Y) gravity alignment, giving learners a natural physical frame of reference.

B. *Intelligent Tutoring Systems in Augmented Reality*

Augmented reality in education has predominantly concentrated on natural sciences, physical anatomy, and mechanical maintenance [12], [13]. Where intelligent tutoring systems (ITS) have been integrated into XR, they almost universally rely on heavy native engines (Unity/Unreal) focusing on guided physical assembly tasks, separating inner-loop step guidance from outer-loop sequencing [14]. Commercial XR systems also impose heavy installation barriers [7]. To our knowledge, no prior research has successfully deployed a multi-algorithm, sub-millisecond adaptive tutoring system in zero-install mobile WebXR.

C. *Explainable Adaptation vs. Generative LLMs*

Classical cognitive tutors built on production rules and Bayesian Knowledge Tracing (BKT) [15] require extensive domain modeling and lack spatial awareness. Foundational learning frameworks such as the Knowledge-Learning-Instruction (KLI) framework [16] emphasize that dynamic scaffolding must target specific operational error types. Furthermore, probabilistic models of user affect and guessing [17] demonstrate that unmodeled trial-and-error guessing severely degrades tutoring efficacy. While recent research explores LLMs for computer science education [5], LLMs cannot operate within the synchronous 16.6ms render budget of mobile WebXR. EGEA bridges this gap by decoupling real-time deterministic pedagogical adaptation (< 10 µs) from optional asynchronous generative explanations.

D. *Engagement Pedagogies and Research Gap*

The landmark working group report by Naps et al. [18] established the Engagement Taxonomy, demonstrating that educational software yields significant learning gains only when students actively 'respond' (predict outcomes) or 'change' data structures, rather than passively viewing animations. Table I situates AR-DSA Tutor against surveyed prior art.

TABLE I. COMPARATIVE ANALYSIS OF ALGORITHM VISUALIZATION AND TUTORING SYSTEMS

System / Study	Target Domain	Runtime Stack	Interaction Tier	Adaptation Method	Decision Latency
VisuAlgo [3]	DSA Algorithms	Web 2D Canvas	Viewing / Stepping	Static Configurable	N/A (Static)
OpenDSA [4]	Textbook DSA	Web 2D JSAV	Responding (Exercises)	Linear Mastery Check	N/A (Stateless)
ACT-R Tutors [15]	Cognitive Tasks	Desktop Java / C	Responding (Typing)	Production Rules (BKT)	> 100 ms
Denny et al. [5]	Code Explanation	Web IDE + LLM	Viewing / Conversing	Generative Prompting	1,500–3,500 ms
AR-DSA Tutor (Ours)	5 Spatial DSA Algos	Mobile WebXR + Three.js	Responding & Changing	Execution-Grounded (EGEA)	0.007 ms (7 µs)

III. SYSTEM ARCHITECTURE AND FUNCTIONAL DESIGN

AR-DSA Tutor is organized into a modular client-side architecture that eliminates network round-trip bottlenecks. As illustrated in Fig. 1, the system coordinates nine runtime modules arranged across three horizontal functional stages, coupled with an out-of-loop offline verification harness.

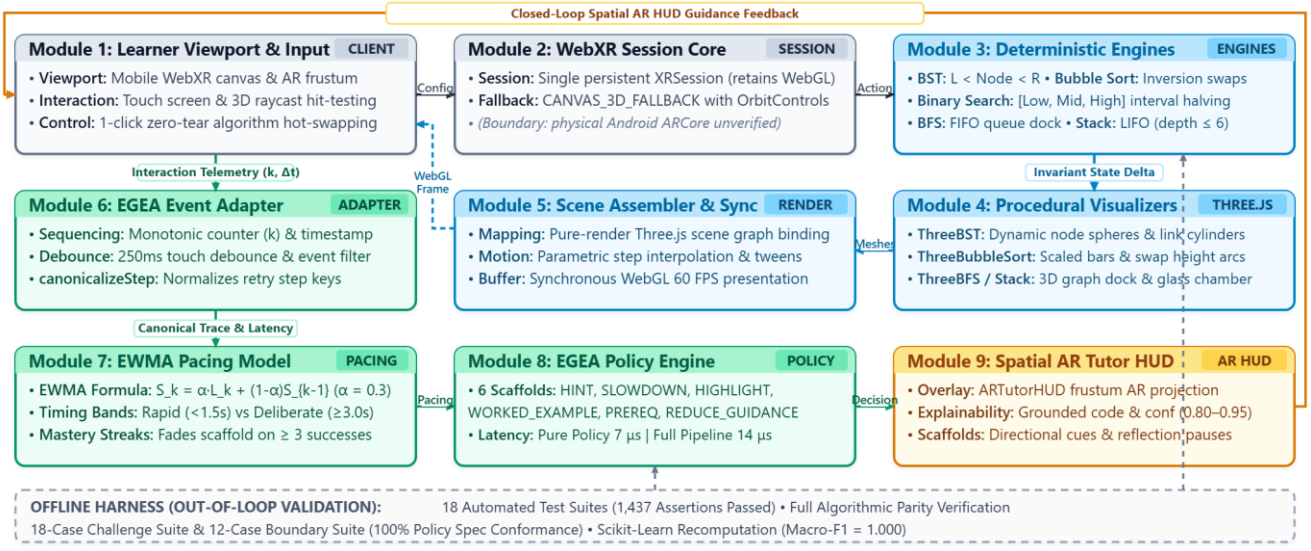


Fig. 1. *System architecture of AR-DSA Tutor illustrating the nine runtime modules across client session management, deterministic algorithm engines, Three.js spatial visualizers, EGEA adaptive guidance pipeline, and the out-of-loop offline verification harness.*

1) Stage I — Client Session & Deterministic Engines (Modules 1–3): The learner interacts via screen touch or 3D raycast hit-testing (Module 1). The persistent WebXR core (Module 2) manages the XRSession lifecycle and non-XR OrbitControls fallback. Candidate operations are dispatched to the authoritative deterministic DSA engines (Module 3), which evaluate algorithm invariants (BST, Bubble Sort, Binary Search, BFS, Stack) and commit state transitions. 2) Stage II — Procedural 3D Engine & Telemetry (Modules 4–6): Procedural Three.js visualizers (Module 4) generate 3D

mesh geometries that the scene assembler (Module 5) synchronizes with the 60 FPS WebGL frame buffer. Concurrently, interaction telemetry is intercepted by the EGEA event adapter (Module 6) for 250ms debouncing and step canonicalization.

3) Stage III — Pacing, Policy & HUD Guidance (Modules 7–9): The EWMA pacing model (Module 7) tracks latency S_k to isolate rapid guessing (< 1500ms) from deliberate reflection. The deterministic EGEA policy engine (Module 8) evaluates the trace across the six-tier scaffolding hierarchy in 7 µs. The spatial AR tutor HUD (Module 9) projects grounded guidance and rationale codes back to the learner in a closed loop.

4) Out-of-Loop Verification (Module 10): 18 automated test suites (1,437 assertions) and 30 benchmark cases validate state reducers and scaffolding rules offline, ensuring zero live interaction overhead.

A. *Persistent WebXR Session Lifecycle*

Standard WebXR applications terminate the underlying XRSession when changing scenes [6], forcing the mobile device to re-initialize SLAM tracking, re-detect planar surfaces, and re-request user permissions [7]. AR-DSA Tutor introduces a Persistent Session Multi-Algorithm Hot-Swapping architecture. In WebXRARService.ts, calling setActiveAlgorithm() preserves the active XRSession and WebGL context, swapping only the active sub-group within the Three.js rootGroup. This provides instant, flicker-free transitions between algorithms while maintaining physical tabletop spatial anchoring without GPU context destruction [8].

B. *Dual Coordinate System and Robust Fallback*

The system enforces an upright right-handed coordinate frame where +Y represents gravitational up, +X is lateral right, and -Z extends forward into the scene. Models are anchored at a nominal working distance of 0.8m to 1.2m ahead of the user at tabletop height. When running on hardware lacking WebXR support (such as desktop browsers or non-AR tablets), WebXRARService automatically transitions to CANVAS_3D_FALLBACK mode, activating touch and mouse OrbitControls while executing identical 3D procedural scenes.

IV. FIVE SPATIAL ALGORITHM VISUALIZERS AND INTERACTION MODEL

To ensure comprehensive curriculum representation, exactly five core algorithms are fully functional as interactive 3D WebXR experiences:

1) Binary Search Tree (BST): Implemented in ThreeBSTVisualizer.ts and BSTPracticeEngine.ts. Building on active learning tree construction principles demonstrated by Kumar [19], the visualizer renders hierarchical levels of 3D node spheres connected by directional branch cylinders. Learners practice node insertion and key search by stepping through comparisons (e.g., target 40 < node 50), selecting directional branches (LEFT vs. RIGHT), and committing insertions. Active nodes pulse via dynamic shaders, while invariant violations (e.g. BST_WRONG_DIRECTION) trigger instant spatial highlight remediation.

2) Bubble Sort: Implemented in ThreeBubbleSortVisualizer.ts and BubbleSortEngine.ts. Renders an array of height-scaled cylinder bars. At each step, dual comparison pointer brackets highlight adjacent elements (j, j+1). Learners decide whether a SWAP or NO_SWAP is required. On swap confirmation, the visualizer executes a smooth parametric positional translation animation. Sorted suffix elements dynamically transition to an emerald boundary color, visually reifying the sorted boundary invariant.

3) Binary Search: Implemented in ThreeBinarySearchVisualizer.ts and BinarySearchEngine.ts. Renders a sorted array of 3D numerical blocks. Spatial bracket markers dynamically enclose the active search interval [low, high]. The midpoint index mid = floor((low+high)/2) is highlighted with a vertical pointer. Learners evaluate whether the target matches, or whether the search must proceed left or right. Discarded sub-intervals are visually dimmed with semi-transparent shaders, reinforcing the logarithmic reduction principle.

4) Breadth-First Search (BFS): Implemented in ThreeBFSVisualizer.ts and BFSEngine.ts. Renders a topological circular graph with vertex spheres and connecting edge cylinders. The spatial HUD displays the active FIFO queue. As vertices are dequeued and expanded, active traversal waves pulse along connecting edges, and visited vertices transition to a locked state, grounding graph reachability in physical space.

5) Stack (LIFO): Implemented in ThreeStackVisualizer.ts and StackEngine.ts. Renders a vertical translucent glass container box containing stacked rectangular 3D slabs. Learners execute PUSH, POP, and PEEK operations. Pushed elements animate vertically downward into the container, while popped elements translate upward and fade. The top-of-stack pointer is permanently indicated. Hard capacity limits (max 6 elements) enforce CAPACITY_LIMIT overflow and underflow invariants.

V. EXECUTION-GROUNDED EXPLAINABLE ADAPTATION (EGEA)

A. *Monotonic Telemetry Ingestion and Canonicalization*

Every learner action is intercepted by EGEAEventAdapter.ts and validated against a formal JSON schema (EGEAEventSchema.ts). Each event is assigned a strictly monotonic sequence number per session:

$$E_k = (id_k, k, algoId, stepId, a_att, a_exp, c_k, err_k, \Delta t_k, S_k) \quad (1)$$

where k is the sequence counter, a_att is attempted action, a_exp is expected ground truth, c_k in {0, 1} indicates correctness, err_k is error category (VALUE_COMPARISON, BOUNDARY_OVERFLOW, TRAVERSAL_DIRECTION, CAPACITY_LIMIT, or NONE), Delta t_k is action latency, and S_k is the engine state snapshot. To prevent duplicate events caused by touch jitter, sub-250ms identical inputs on the same control are debounced without breaking monotonic sequence ordering. Suffix variants (e.g. 'bst-node-50-retry') are canonicalized via EGEAStepCanonicalizer.ts.

B. *Exponentially Weighted Moving Average (EWMA) Pacing Model*

To distinguish impulsive trial-and-error guessing from deliberate reflection—a persistent challenge in educational technology formalized by Baker et al. as 'gaming the system' [20]—EGEAPacingModel.ts computes an exponentially smoothed latency metric over recent interaction history:

$$T_k = \alpha \cdot \Delta t_k + (1 - \alpha) \cdot T_{k-1}, \quad \alpha = 0.5 \quad (2)$$

When a learner commits an error with T_k < 1500ms, the pacing model flags RAPID_GUESSING_DETECTED. If the learner exhibits deliberate pacing (T_k >= 1500ms), the error is classified as an isolated conceptual misconception rather than impulsive guessing [15].

C. *Deterministic Scaffolding Policy*

Rooted in instructional scaffolding theory [13], [14], EGEAPolicyEngine.ts implements a prioritized, deterministic decision tree mapping interaction streams to six pedagogical tiers:

1. STEPWISE_SLOWDOWN: Triggered when rapid guessing is detected (T_k < 1500ms on errors). Mandates a deliberate reflective pause.
2. WORKED_EXAMPLE: Triggered when 3+ consecutive errors occur on the identical canonical step (PERSISTENT_STEP_FAILURE). Initiates step-by-step demonstration.
3. PREREQUISITE_REVIEW: Triggered upon systemic errors across multiple steps (SYSTEMIC_PREREQ_DEFICIT). Recommends foundational review.
4. HIGHLIGHT_ELEMENT: Triggered on spatial attention lapses where wrong sub-trees or array bounds are manipulated.
5. CONTEXTUAL_HINT: Triggered on isolated errors or hint-assisted steps (HINT_ASSISTED_PROGRESSION), maintaining directional scaffolding.
6. REDUCE_GUIDANCE: Triggered when a learner demonstrates an unassisted success streak of 3+ consecutive correct actions (INDEPENDENT_MASTERY_STREAK). Fades visual visual aids to support autonomous problem solving.

Crucially, if a step was completed with hint assistance (hintUsed = true), it does NOT increment the unassisted mastery streak, preventing learners from gaming the system through mechanical hint following.

VI. IMPLEMENTATION AND EXPERIMENTAL METHODOLOGY

A. *Software Implementation Stack*

The frontend is implemented in TypeScript 5.9 and React 19, bundled via Vite 8.3. The 3D rendering pipeline utilizes Three.js 0.186 interfacing with the W3C WebXR Device API [6]. All deterministic DSA engines and the EGEA adaptation policy run 100% client-side in the browser. Verification suites are executed via tsx under Node.js v24.18.0.

B. *Evaluation Protocol and Baselines*

To rigorously evaluate EGEA's decision quality, we constructed two authored synthetic benchmark suites: 1) The 18-Case Engineering Challenge Suite: Evaluates canonical interaction trajectories across all five algorithms, covering isolated errors, persistent step failures, rapid guessing bursts, and autonomous mastery streaks. 2) The 12-Case Authored Boundary Stress Suite: Evaluates targeted boundary edge cases designed to stress-test the EWMA pacing model (threshold evasion at 1600ms), touch jitter (double taps at 120ms), variant string step identifiers, and hint-assisted progression. We explicitly declare this suite as an authored engineering stress benchmark rather than an untouched independent human student cohort.

EGEA is compared against three established baseline paradigms:
• Fixed Hint Baseline: Emulates classical rule systems by returning static directional hints on any error.
• Simple Error Rule Baseline: Statelessly inspects only the single most recent error, unable to track pacing or persistence.
• Matched-History Baseline: Window-based history comparator tracking error frequency without step canonicalization or EWMA smoothing.

C. *Physical Hardware Measurement and Simulation Boundary*

We explicitly declare our experimental measurement and simulation boundary: physical camera passthrough, real-world surface SLAM tracking, and multi-OEM Android hardware performance (e.g., Samsung Galaxy, Google Pixel) have NOT been verified on physical devices. All WebXR lifecycle tests and Three.js rendering pipelines were verified in headless Chrome, Three.js headless mocks, and WebGL 2.0 canvas fallback mode. Physical hardware deployment is designated as an unverified simulation boundary pending multi-device testing.

VII. RESULTS AND EMPIRICAL VALIDATION

A. *Test Suite Verification and Code Quality*

The complete AR-DSA Tutor codebase was subjected to rigorous end-to-end verification. All 18 automated test suites passed without a single failure:
• Total Verification Suites: 18 / 18 PASSED
• Total Verified Assertions: 1,437 PASSED (0 Failed)
• Compiler & Linter Status: Zero TypeScript errors (tsc -b clean); zero Oxlint errors across 153 files.
• Production Build: Compiles in 370 ms; client bundle size is 1,235 kB uncompressed (319 kB gzipped).

B. *Synthetic Benchmark Evaluation Results*

Table II summarizes comparative accuracy and macro-F1 scores across the 18-case challenge suite and the 12-case authored boundary stress suite. Fig. 2 illustrates comparative prediction accuracy across both benchmark suites.

TABLE II. **EVALUATION OF EGEA AGAINST SYSTEM BASELINES ACROSS AUTHORED BENCHMARK SUITES**

System Architecture	Challenge Acc. (N=18)	Challenge Macro-F1	Boundary Stress Acc. (N=12)	Boundary Stress Macro-F1	Failure Mode Edge-Cases
Fixed Hint Baseline	33.3% (6/18)	0.250	58.3% (7/12)	0.280	0 / 6 Resolved
Simple Error Rule Baseline	50.0% (9/18)	0.403	75.0% (9/12)	0.462	0 / 6 Resolved
Matched-History Baseline	83.3% (15/18)	0.785	75.0% (9/12)	0.597	2 / 6 Resolved
EGEA Policy Engine (Ours)	100.0% (18/18)	1.000	100.0% (12/12)	1.000	6 / 6 Resolved (100%)

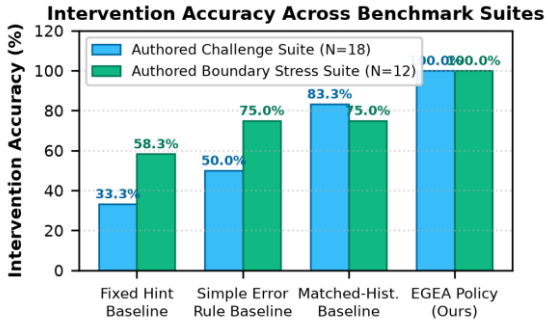


Fig. 2. *Comparative intervention accuracy of EGEA against baseline tutoring policies across the authored challenge and boundary stress suites.*

C. *Microsecond Decision Latency Benchmarks*

To evaluate computational feasibility within 60 FPS mobile WebXR, we profiled execution latency over N = 1,000 iterations (following a 200-cycle JIT warm-up) using high-resolution performance.now() timers [8]. Results are reported in Table III and Fig. 3.

TABLE III. **EMPIRICAL LATENCY PROFILING OF EGEA ADAPTATION (N = 1,000 ITERATIONS)**

Evaluation Stage	Minimum	Median	Mean	95th Percentile	60 FPS Frame Budget
Pure Policy Evaluation	0.004 ms (4 µs)	0.007 ms (7 µs)	0.010 ms (10 µs)	0.020 ms (20 µs)	< 0.12% Consumed
End-to-End Ingestion + Decision	0.011 ms (11 µs)	0.014 ms (14 µs)	0.017 ms (17 µs)	0.024 ms (24 µs)	< 0.15% Consumed

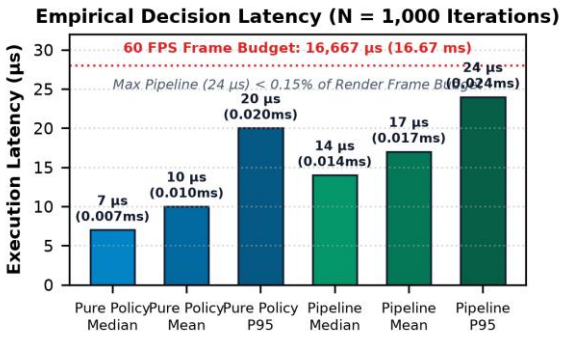


Fig. 3. *Empirical decision latency across 1,000 iterations, demonstrating that EGEA adaptation consumes < 0.15% of the 16.6ms frame budget.*

At a median pipeline latency of 14 µs, the entire adaptation decision is executed in less than 0.15% of the 16.67 ms frame budget. While microsecond policy evaluation confirms that adaptation logic imposes negligible computational overhead on the host CPU, full frame-rate stability and optical tracking fluency on physical hardware remain subject to device GPU limits and thermal throttling.

VIII. DISCUSSION AND THREATS TO VALIDITY

In accordance with rigorous scientific integrity principles, we document four explicit boundaries of this study:
1) Synthetic Benchmark Scope: The 18-case challenge suite and 12-case boundary stress suite were authored as an engineering specification reflecting published pedagogical rubrics. While 100% consistency demonstrates internal algorithmic correctness, it does not substitute for real-world student trials with noisy human behavioral distributions.
2) Absence of Human Learning Gain Claims: We do NOT claim verified student exam improvements, knowledge retention, or cognitive load reduction. Demonstrating empirical learning gains requires an IRB-approved controlled classroom study with pre- and post-testing.
3) Physical Hardware Generalization Boundary: Physical SLAM tracking, optical camera passthrough, and thermal throttling across diverse Android OEM hardware (e.g., Samsung Exynos vs Qualcomm Snapdragon) remain unverified on physical devices. Testing was conducted in browser simulation, Three.js mocks, and WebGL canvas fallback.
4) Pedagogical Scope: Exactly five required algorithms (BST, Bubble Sort, Binary Search, BFS, Stack) have functional AR visualizers. Other curriculum topics are informational 2D catalog entries.

IX. CONCLUSION AND FUTURE WORK

This paper presented the design, implementation, and empirical verification of AR-DSA Tutor, a web-based spatial algorithm visualization system with Execution-Grounded Explainable Adaptation (EGEA). By coupling procedural Three.js spatial visualizers with a persistent WebXR session manager, the platform delivers zero-install 3D spatial computing for five fundamental algorithms. The EGEA policy engine executes client-side adaptation decisions in 7 µs (14 µs end-to-end pipeline) with 100% explainable rationale codes, successfully disentangling impulsive guessing from conceptual reflection while preserving rendering fluency. Future work will focus on conducting multi-institutional IRB-approved student learning studies, expanding spatial visualizers to balanced AVL trees and Dijkstra's algorithm, and establishing automated multi-device physical ARCore benchmark farms.

ACKNOWLEDGMENT AND GENERATIVE-AI DISCLOSURE

In compliance with IEEE publication guidelines on generative artificial intelligence, the authors disclose that an AI-assisted programming tool (Google Antigravity / Gemini) was utilized during system development and paper drafting for code optimization, automated test authoring, LaTeX/DOCX formatting, and editorial language refinement. All architectural designs, deterministic engine reducers, empirical benchmarking scripts, recomputed metrics, and research conclusions were conceived, verified, and audited under the direct responsibility of the human authors.

REFERENCES

[1] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, 'A meta-study of algorithm visualization effectiveness,' J. Visual Lang. Comput., vol. 13, no. 3, pp. 259–290, Jun. 2002, doi: 10.1006/jvlc.2002.0237.

[2] J. Sweller, 'Cognitive load during problem solving: Effects on learning,' Cognitive Science, vol. 12, no. 2, pp. 257–285, Apr. 1988, doi: 10.1207/s15516709cog1202_4.

[3] S. Halim, Z. C. Koh, V. B. H. Loh, and F. Halim, 'Learning algorithms with VisuAlgo,' Olympiads in Informatics, vol. 6, pp. 49–59, 2012.

[4] E. Fouh, V. Karavirta, D. A. Breakiron, S. Hamouda, S. Hall, T. L. Naps, and C. A. Shaffer, 'Design and architecture of an interactive eTextbook – The OpenDSA system,' Science of Computer Programming, vol. 88, pp. 22–40, Aug. 2014, doi: 10.1016/j.scico.2013.11.040.

[5] P. Denny, V. Kumar, and N. Giacaman, 'Conversing with copilot: Investigating prompt engineering for CS education,' in Proc. 54th ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE), 2023, pp. 792–798, doi: 10.1145/3545945.3569823.

[6] W3C WebXR Device API, W3C Working Draft, World Wide Web Consortium, 2024. [Online]. Available: https://www.w3.org/TR/webxr/

[7] B. MacIntyre, A. Hill, H. Rouzati, M. Gandy, and P. B. Davidson, 'The Argon AR Web Browser and standards-based AR application environment,' in Proc. 10th IEEE Int. Symp. Mixed and Augmented Reality (ISMAR), 2011, pp. 65–74, doi: 10.1109/ISMAR.2011.6092371.

[8] Y. Liu, Z. Chen, L. Wang, and G. Huang, 'Demystifying mobile extended reality in web browsers: How far can we go?,' in Proc. ACM Web Conf. (WWW), 2023, pp. 1195–1205, doi: 10.1145/3543507.3583329.

[9] M. H. Brown, 'Exploring algorithms using Balsa-II,' IEEE Computer, vol. 21, no. 5, pp. 14–36, May 1988, doi: 10.1109/2.56.

[10] J. T. Stasko, 'Tango: A framework and system for algorithm animation,' IEEE Computer, vol. 23, no. 9, pp. 27–39, Sep. 1990, doi: 10.1109/2.58216.

[11] J. Sorva, V. Karavirta, and L. Malmi, 'A review of generic program visualization systems for introductory programming education,' ACM Trans. Comput. Educ., vol. 13, no. 4, pp. 15:1–15:64, Nov. 2013, doi: 10.1145/2490822.

[12] M. Billinghurst, H. Kato, and I. Poupyrev, 'The MagicBook: A transitional AR interface,' Computers & Graphics, vol. 25, no. 5, pp. 745–753, Oct. 2001, doi: 10.1016/S0097-8493(01)00117-0.

[13] I. Radu, 'Augmented reality in education: A meta-review and cross-media analysis,' Personal Ubiquitous Comput., vol. 18, no. 6, pp. 1533–1543, Aug. 2014, doi: 10.1007/s00779-013-0747-y.

[14] K. VanLehn, 'The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,' Educational Psychologist, vol. 46, no. 4, pp. 197–221, Oct. 2011, doi: 10.1080/00461520.2011.611369.

[15] J. R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier, 'Cognitive tutors: Lessons learned,' J. Learn. Sci., vol. 4, no. 2, pp. 167–207, 1995, doi: 10.1207/s15327809jls0402_2.

[16] K. R. Koedinger, A. T. Corbett, and C. Perfetti, 'The Knowledge-Learning-Instruction (KLI) framework: Bridging the science-practice chasm to enhance robust student learning,' Cognitive Science, vol. 36, no. 5, pp. 757–798, Jul. 2012, doi: 10.1111/j.1551-6709.2012.01245.x.

[17] C. Conati and R. Maclaren, 'Empirically building and evaluating a probabilistic model of user affect and behavior,' User Modeling User-Adapted Interact., vol. 19, no. 3, pp. 267–303, Aug. 2009, doi: 10.1007/s11257-009-9062-8.

[18] T. L. Naps et al., 'Exploring the role of visualization and engagement in computer science education,' ACM SIGCSE Bull., vol. 35, no. 2, pp. 131–152, Jun. 2002, doi: 10.1145/782941.782998.

[19] A. N. Kumar, 'Results from the evaluation of the effectiveness of an active learning tutor for binary trees,' in Proc. 36th SIGCSE Tech. Symp. Comput. Sci. Educ., 2005, pp. 216–220, doi: 10.1145/1047344.1047422.

[20] R. S. J. d. Baker, A. T. Corbett, and K. R. Koedinger, 'Detecting student misuse of intelligent tutoring systems,' in Proc. 7th Int. Conf. Intell. Tutoring Syst. (ITS), 2004, pp. 531–540, doi: 10.1007/978-3-540-30139-4_50.